



Practica 2

[Practica-2---Semforos.pdf](#)

CONSIDERACIONES PARA RESOLVER LOS EJERCICIOS:

- Los semáforos deben estar declarados en todos los ejercicios.
 - Los semáforos deben estar inicializados en todos los ejercicios.
 - No se puede utilizar ninguna sentencia para setear o ver el valor de un semáforo.
 - Debe evitarse hacer busy waiting en todos los ejercicios.
 - En todos los ejercicios el tiempo debe representarse con la función delay.
1. **Existen N personas que deben ser chequeadas por un detector de metales antes de poder ingresar al avión.**
 - a. **Analice el problema y defina qué procesos, recursos y semáforos/sincronizaciones serán necesarios/convenientes para resolverlo.**
 - b. **Implemente una solución que modele el acceso de las personas a un detector (es decir, si el detector está libre la persona lo puede utilizar; en caso contrario, debe esperar).**

```
sem mutex = 1;
Process persona [id: 0..N]{
    P(mutex);
    //usa detector de metales
    V(mutex);
}
```

- c. **Modifique su solución para el caso que haya tres detectores.**

```
sem detector = 3;
Process persona [id: 0..N]{
    P(detector);
    //usa detector de metales
    V(detector);
}
```

- d. **Modifique la solución anterior para el caso en que cada persona pueda pasar más de una vez, siendo aleatoria esa cantidad de veces.**

```
sem detector = 3;
Process persona [id: 0..N]{
    for i: 1 to Random(N){
        P(detector);
        //usa detector
        V(detector);
    }
}
```

2. **Un sistema de control cuenta con 4 procesos que realizan chequeos en forma colaborativa. Para ello, reciben el historial de fallos del día anterior (por simplicidad, de tamaño N). De cada fallo, se conoce su número de identificación (ID) y su nivel de gravedad (0=bajo, 1=intermedio, 2=alto, 3=crítico). Resuelva considerando las siguientes situaciones:**

- a. **Se debe imprimir en pantalla los ID de todos los errores críticos (no importa el orden).**

```
Fallos historial[N]
int buscar = N div 4;

Process buscador [id: 0..N-1]{
    int inicio = id * buscar;
    int fin = (ini + buscar - 1);
    for i:= inicio .. fin {
```

```

        if (historial[i].gravedad == 3){
            print(hitorial[i].id);
        }
    }
}

```

- b. Se debe calcular la cantidad de fallos por nivel de gravedad, debiendo quedar los resultados en un vector global.

```

Fallos historial[4]
int buscar = N div 4;
int cantidadFallos [4] = ([4],0);
sem mutex = 1;

Process buscador [id: 0..N-1]{
    int cantidadFallosLocal [4] = ([4],0);
    int inicio = id * buscar;
    int fin = (ini + buscar - 1);
    for i:= inicio.. fin {
        cantidadFallosLocal[hitorial[i].gravedad]++;
    }
    P(mutex);
    for j:= 0..3 {
        cantidadFallos[j] = cantidadFallos + cantidad
        FallosLocal[j];
    }
    V(mutex);
}

```

- c. Ídem b) pero cada proceso debe ocuparse de contar los fallos de un nivel de gravedad determinado.

```

Fallos historial[4]
int cantidadFallos [4] = ([4],0);

Process buscador [id: 0..N-1]{
    int cant = 0;
    for i: 0.. N-1{

```

```

        if (historial[i].gravedad == id){
            cantidadFallos[id] = cantidadFallos[id] +
1;
        }
    }

```

3. Un sistema operativo mantiene 5 instancias de un recurso almacenadas en una cola. Además, existen P procesos que necesitan usar una instancia del recurso. Para eso, deben sacar la instancia de la cola antes de usarla. Una vez usada, la instancia debe ser encolada nuevamente para su reúso.

```

Cola c;
sem mutex = 1;
sem instancias = 5;

Process P [id: 0.. P-1]{
    P(mutex);
    P(instancias)
    r = c.pop();
    V(mutex);
    // uso recurso
    P(mutex)
    c.push(r)
    V(mutex)
    V(instancias)
}

```

4. Suponga que existe una BD que puede ser accedida por 6 usuarios como máximo al mismo tiempo. Además, los usuarios se clasifican como usuarios de prioridad alta y usuarios de prioridad baja. Por último, la BD tiene la siguiente restricción:
- no puede haber más de 4 usuarios con prioridad alta al mismo tiempo usando la BD.
 - no puede haber más de 5 usuarios con prioridad baja al mismo tiempo usando la BD.

Indique si la solución presentada es la más adecuada. Justifique la respuesta.

Var total: sem := 6; alta: sem := 4; baja: sem := 5;	
Process Usuario-Alta [I:1..L]:: { P (total); P (alta); <i>// usa la BD</i> V(total); V(alta); }	Process Usuario-Baja [I:1..K]:: { P (total); P (baja); <i>// usa la BD</i> V(total); V(baja); }

-La solución no es correcta porque hace P(total) antes de hacer P(alta) en ambos tipos de Usuario. Lo que genera esto es que los procesos que puedan acceder porque cumplen la condición terminan restringidos cuando no se debería, por ejemplo, si hay 5 de prioridad alta y el 5to hace P(total) estarían habiendo 5 usuarios de prioridad alta en simultáneo y eso no se puede porque rompe con la condición y no estaría dejando que los usuarios de prioridad Baja accedan y terminen restringidos

5. En una empresa de logística de paquetes existe una sala de contenedores donde se preparan las entregas. Cada contenedor puede almacenar un paquete y la sala cuenta con capacidad para N contenedores. Resuelva considerando las siguientes situaciones:

- a. La empresa cuenta con 2 empleados: un empleado Preparador que se ocupa de preparar los paquetes y dejarlos en los contenedores; un empleado Entregador que se ocupa de tomar los paquetes de los contenedores y realizar las entregas. Tanto el Preparador como el Entregador trabajan de a un paquete por vez.

```
buffer[n];
int ocupado = 0, libre = 0;
sem vacio = N, lleno = 0
```

```
Process Preparador {
    while (true){
```

```

        P(vacio); // Espero que haya algun lugar vaci
o
        Paquete p; // Preparo paquete
        buffer[libre] = p; // Lo pongo
        libre = (libre + 1) MOD N; // Calculo la prox
ima posicion libre
        V(lleno) // Aviso al Entregador que hay algo
en el contenedor
    }
}

Process Entregador {
    while(true){
        P(lleno); // Espero que haya algo en el conte
nedor
        entrega = buffer[ocupado]; // Saco el paquete
        ocupado = (ocupado + 1) MOD N // Calculo prox
ima posicion ocupada
        V(vacio) // Aumento la cantidad de espacio
    }
}

```

- b. **Modifique la solución a) para el caso en que haya P empleados Preparadores.**

```

buffer[n];
int ocupado = 0, libre = 0;
sem vacio = N, lleno = 0
sem mutexP = 1;

Process Preparador [id: 0..P-1] {
    while (true){
        P(vacio); // Espero que haya algun lugar vaci
o
        P(mutexP);
        Paquete p; // Preparo paquete
        buffer[libre] = p; // Lo pongo
        libre = (libre + 1) MOD N; // Calculo la prox

```

```

ima posicion libre
    V(mutexP);
    V(lleno) // Aviso al Entregador que hay algo
en el contenedor
}
}

Process Entregador {
    while(true){
        P(lleno); // Espero que haya algo en el conte
nedor
        entrega = buffer[ocupado]; // Saco el paquete
        ocupado = (ocupado + 1) MOD N // Calculo prox
ima posicion ocupada
        V(vacio) // Aumento la cantidad de espacio
    }
}

```

- c. **Modifique la solución a) para el caso en que haya E empleados Entregadores.**

```

buffer[n];
int ocupado = 0, libre = 0;
sem vacio = N, lleno = 0
sem mutexE = 1;

Process Preparador {
    while (true){
        P(vacio); // Espero que haya algun lugar vaci
o
        Paquete p; // Preparo paquete
        buffer[libre] = p; // Lo pongo
        libre = (libre + 1) MOD N; // Calculo la prox
ima posicion libre
        V(lleno) // Aviso al Entregador que hay algo
en el contenedor
    }
}

```

```

Process Entregador [id: 0.. E-1] {
    while(true){
        P(lleno); // Espero que haya algo en el contenedor
        P(mutexE);
        entrega = buffer[ocupado]; // Saco el paquete
        ocupado = (ocupado + 1) MOD N // Calculo proxima posicion ocupada
        V(mutexE);
        V(vacio) // Aumento la cantidad de espacio
    }
}

```

- d. **Modifique la solución a) para el caso en que haya P empleados Preparadores y E empleados Entregadores.**

```

buffer[n];
int ocupado = 0, libre = 0;
sem vacio = N, lleno = 0
sem mutexP = 1, mutexE = 1;

Process Preparador [id: 0..P-1] {
    while (true){
        P(vacio); // Espero que haya algun lugar vacio
        P(mutexP);
        Paquete p; // Preparo paquete
        buffer[libre] = p; // Lo pongo
        libre = (libre + 1) MOD N; // Calculo la proxima posicion libre
        V(mutexP);
        V(lleno) // Aviso al Entregador que hay algo en el contenedor
    }
}

Process Entregador [id: 0.. E-1] {
    while(true){

```

```

        P(lleno); // Espero que haya algo en el conte
nedor
        P(mutexE);
        entrega = buffer[ocupado]; // Saco el paquete
        ocupado = (ocupado + 1) MOD N // Calculo prox
ima posicion ocupada
        V(mutexE);
        V(vacio) // Aumento la cantidad de espacio
    }
}

```

6. Existen N personas que deben imprimir un trabajo cada una. Resolver cada ítem usando semáforos:

- a. Implemente una solución suponiendo que existe una única impresora compartida por todas las personas, y las mismas la deben usar de una persona a la vez, sin importar el orden. Existe una función `Imprimir(documento)` llamada por la persona que simula el uso de la impresora. Sólo se deben usar los procesos que representan a las Personas.

```

sem mutex = 1;

Process Persona [id: 0..N-1]{
    Documento d;
    P(mutex);
    Imprimir(documento);
    V(mutex);
}

```

- b. Modifique la solución de (a) para el caso en que se deba respetar el orden de llegada.

```

sem mutex = 1 , espera[N] ([N],0)
Cola c; // colaEspecial c
boolean libre = true;

```

```

Process Persona [id: 0..N-1]{
    Documento d;
    int aux;
    P(mutex);
    if (libre){
        libre = false;
        V(mutex);
    } else{
        c.push(id);
        V(mutex);
        P(espera[id]);
    }
    Impimir(documento); // Uso recurso
    P(mutex);
    if (c.isEmpty()) { libre = true}
    else {
        c.pop(aux);
        V(espera[aux]);
    }
    V(mutex);
}

```

- c. Modifique la solución de (a) para el caso en que se deba respetar estrictamente el orden dado por el identificador del proceso (la persona X no puede usar la impresora hasta que no haya terminado de usarla la persona X-1).

```

sem semaforo [N] = ([N],0)

Process Persona [id: 0..N1]{
    if (id == 0){
        Imprimir();
    } else {
        P(semaforo[id-1]);
        Imprimir();
    }
    V(semaforo[id]);
}

```

- d. **Modifique la solución de (b) para el caso en que además hay un proceso Coordinador que le indica a cada persona que es su turno de usar la impresora.**

```
cola c;
sem mutex = 1, llegada = 0, listo = 0, espera[N] (N, 0);

Process Persona [id: 0.. N-1]{
    Documento d;
    P(mutex);
    c.push(id); // Llego y me encolo
    V(mutex);
    V(llegada); // Aviso que llegue
    P(espera[id]); // Espero a que me llame Coordinador
or
    Imprimir(documento);
    V(listo); // Aviso que termine
}

Process Coordinador{
    int id;
    for i: 0.. N-1 {
        P(llegada); // Espero que llegue alguien
        P(mutex);
        c.pop(id); // Saco a uno de la cola
        V(mutex);
        V(espera[id]); // Lo despierto
        P(listo); // Decremento listo para esperar que otro termine y lo aumente
    }
}
```

- e. **Modificar la solución (d) para el caso en que sean 5 impresoras. El coordinador le indica a la persona cuándo puede usar una impresora, y cual debe usar.**

```

cola c;
sem mutex = 1, mutexImpl = 1, llegada = 0, espera[N]
(N,0); impLibres = 5
boolean libre[5] = ([5], true); int impAsignada[N]
([N], -1);

Process Persona [id: 0.. N-1]{
    int imp;
    Documento d;
    P(mutex);
    c.push(id); // Llego y me encolo
    V(mutex);
    V(llegada); // Aviso que llegue
    P(espera[id]); // Espero aviso de coordinador
    imp = impAsignada[id]; // Agarro que impresora usar
    Imprimir(documento, imp); // USO RECURSO
    P(mutexImpl); // Me fijo si no hay nadie usando array de libres
    libre[imp] = true; // Marco impresora nuevamente como libre
    V(impLibres); // Libero array de libres
    V(mutexImpl); // Libero array de impresoras
}

Process Coordinador{
    int id,i,j;
    for i: 0.. N-1{
        P(llegada); // Espero llegada
        P(impLibres); // Espero array de libres
        P(mutex);
        c.pop(id); // Saco a uno de la cola
        V(mutex);
        P(mutexImpl) // Me fijo si no hay nadie usando array de libres
        j = 0;
        while (not libre[j]) { j++; } // Busco impresora libre
    }
}

```

```

        libre[j] = false;
        V(mutexImpl); // Libero impresora de libres
        impAsignada[id] = j; // Asigno impresora libre
        e al id especifico
        V(espera[id]); // Despierto al id especifico
    }
}

```

7. Suponga que se tiene un curso con 50 alumnos. Cada alumno debe realizar una tarea y existen 10 enunciados posibles. Una vez que todos los alumnos eligieron su tarea, comienzan a realizarla. Cada vez que un alumno termina su tarea, le avisa al profesor y se queda esperando el puntaje del grupo (depende de todos aquellos que comparten el mismo enunciado). Cuando un grupo termina, el profesor les otorga un puntaje que representa el orden en que se terminó esa tarea de las 10 posibles.

Nota: Para elegir la tarea, suponga que existe una función elegir que le asigna una tarea a un alumno (esta función asignará 10 tareas diferentes entre 50 alumnos, es decir, que 5 alumnos tendrán la tarea 1, otros 5 la tarea 2 y así sucesivamente para las 10 tareas).

```

int cant = 0, N = 50
int resultados[10];
sem mutex = 1, mutexC = 1, barrera = 0, aviso = 0, grupos[10] = ([10], 0);
Cola c;

Process Alumno [id = 0..N-1]{
    int tarea, resultado;
    elegir(tarea);
    P(mutex):
    cant++; // Llego y aumento cant de alumnos que eligieron tarea
    if (cant == 50) { for i:1..N-1 { V(barrera)} } // Despierto a los dormidos
    V(mutex);
    P(barrera); // Espero en barrera a que todos elijan 1 tarea
}

```

```

// Hago tarea
P(mutexC);
c.push(tarea); // Indico que termine la tarea X
V(mutexC);
V(avisos); // Despierto al profesor para avisarle que
termine

P(grupos[tarea]); // Me quedo esperando hasta entreg
a de correccion
resultado = resultados[tarea]; // Agarro el resultad
o
}

Process Profesor{
    int tarea, puesto = 0;
    int terminados[10] = ([10],0);
    for i:= 0 ... N-1{
        P(avisos); // Espero a que alumno termine tarea
        P(mutexC);
        c.pop(tarea); // Saco la tarea que termino
        V(mutexC);
        // Corrige
        terminados[tarea] = terminados[tarea] + 1;
        if (terminados[tarea] == 5) {
            // Si hay 5 significa que todos los alumnos
de la tarea X la hicieron
            resultados[tarea] = puesto; // Asigno el pue
sto de fin como resultado
            for i: 0 .. 4 { V(grupos[tarea]) } // Despie
rto para que vean resultados
            puesto = puesto + 1;
        }
    }
}

```

8. Una fábrica de piezas metálicas debe producir T piezas por día. Para eso, cuenta con E empleados que se ocupan de producir las piezas de a una

por vez. La fábrica empieza a producir una vez que todos los empleados llegan. Mientras haya piezas por fabricar, los empleados tomarán una y la realizarán. Cada empleado puede tardar distinto tiempo en fabricar una pieza. Al finalizar el día, se debe conocer cuál es el empleado que más piezas fabricó.

- Implemente una solución asumiendo que $T > E$.
- Implemente una solución que contemple cualquier valor de T y E .

```
int cant = 0, max = -1, maxEmp = -1;
sem barrera = 0, mutex = 1, mutexT = 1, mutexMax = 1
piezas = T

Process Empleado [0.. E-1]{
    int cantPiezas = 0;
    P(mutex);
    cant++;
    if (cant == E) { for i: 0.. E-1 { V(barrera)} } // S
i todos llegaron, despierto
    V(mutex);
    P(barrera); // Espero en barrera a que me despierten

    P(mutexT)
    while (piezas > 0){
        piezas--;
        V(mutexT);
        //Produzco pieza
        cantPiezas++;
        P(mutexT);
    }
    // No hay mas piezas, libero y calculo si mis piezas
son mas que Max
    V(mutexT);
    P(mutexMax);
    if (cantPiezas > max){
        max = cantPiezas;
        maxEmp = id;
    }
}
```

```
V(mutexMax) ;  
}
```

9. Resolver el funcionamiento en una fábrica de ventanas con 7 empleados (4 carpinteros, 1 vidriero y 2 armadores) que trabajan de la siguiente manera:

- Los carpinteros continuamente hacen marcos (cada marco es armado por un único carpintero) y los dejan en un depósito con capacidad de almacenar 30 marcos.
- El vidriero continuamente hace vidrios y los deja en otro depósito con capacidad para 50 vidrios.
- Los armadores continuamente toman un marco y un vidrio (en ese orden) de los depósitos correspondientes y arman la ventana (cada ventana es armada por un único armador).

10. A una cerealera van T camiones a descargarse trigo y M camiones a descargar maíz. Sólo hay lugar para que 7 camiones a la vez descarguen, pero no pueden ser más de 5 del mismo tipo de cereal.

- a. Implemente una solución que use un proceso extra que actúe como coordinador entre los camiones. El coordinador debe atender a los camiones según el orden de llegada. Además, debe retirarse cuando todos los camiones han descargado.
- b. Implemente una solución que no use procesos adicionales (sólo camiones). No importa el orden de llegada para descargar. Nota: maximice la concurrencia.

11. En un vacunatorio hay un empleado de salud para vacunar a 50 personas. El empleado de salud atiende a las personas de acuerdo con el orden de llegada y de a 5 personas a la vez. Es decir, que cuando está libre debe esperar a que haya al menos 5 personas esperando, luego vacuna a las 5 primeras personas, y al terminar las deja ir para esperar por otras 5. Cuando ha atendido a las 50 personas el empleado de salud se retira.

Nota: todos los procesos deben terminar su ejecución; suponga que el empleado tiene una función `VacunarPersona()` que simula que el empleado

está vacunando a UNA persona.

12. Simular la atención en una Terminal de Micros que posee 3 puestos para hisopar a 150 pasajeros. En cada puesto hay una Enfermera que atiende a los pasajeros de acuerdo con el orden de llegada al mismo. Cuando llega un pasajero, se dirige al Recepcionista, quien le indica qué puesto es el que tiene menos gente esperando. Luego se dirige al puesto y espera a que la enfermera correspondiente lo llame para hisoparlo. Finalmente, se retira.

- a. Implemente una solución considerando los procesos Pasajeros, Enfermera y Recepcionista.**
- b. Modifique la solución anterior para que sólo haya procesos Pasajeros y Enfermera, siendo los pasajeros quienes determinan por su cuenta qué puesto tiene menos personas esperando.**

Nota: suponga que existe una función Hisopar() que simula la atención del pasajero por parte de la enfermera correspondiente.